

RECURSION AND BACKTRACKING

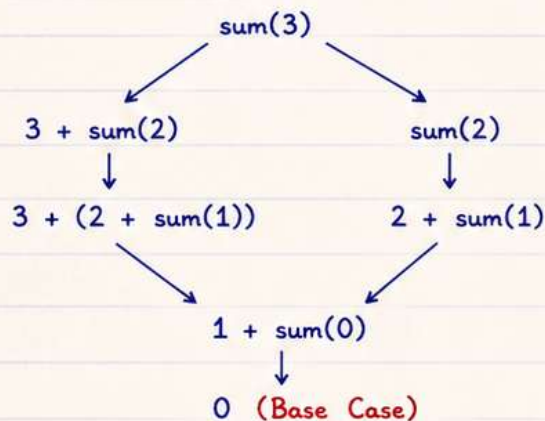
1. Understanding Recursion

- Recursion is a technique where a function calls itself to solve smaller instances of the same problem.
- It breaks a problem into smaller sub-problems of the same type.
- It continues until a **base condition** is met.

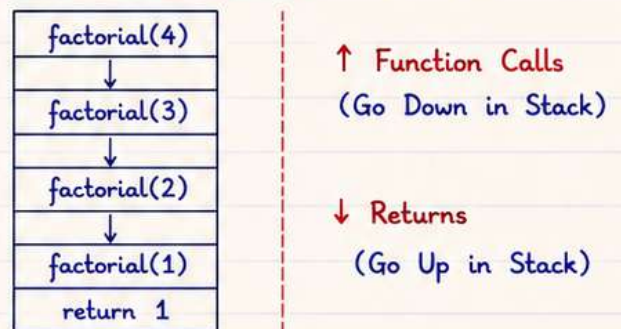
Base Case vs Recursive Case

- **Base Case** → The condition when recursion stops.
- **Recursive Case** → The condition when the function calls itself with a smaller input.

Recursive Tree (Example: sum(3))



Recursion Call Stack (Example: factorial(4))



General Recursion Template (Java)

```
returnType function(parameters) {  
    // 1. Base Case  
    if (base condition) {  
        return base_value;  
    }  
    // 2. Recursive Case  
    // Do some work  
    return function(smaller_input);  
}
```

Recursive Flow



Common Mistakes

- Missing Base Case → Leads to infinite recursion.
- Wrong Base Case → Gives wrong result.
- Not reducing the input → Leads to infinite calls.
- Stack Overflow → For very deep recursion.
- Not returning the value properly.

Quick Summary

- Recursion solves a problem by calling itself with a smaller input.
- Every recursive solution must have a base case.
- Think of recursion as a call stack (go down for calls, up for returns).
- Use recursion only when it makes the solution simpler and clearer.

2. Basic Recursion Problems

These problems help to build strong foundation for solving complex recursion questions.

1. Factorial of N

Example / Dry Run

$n = 4$
 $fact(4) = 4 * fact(3)$
 $\downarrow$
 $= 4 * 3 * fact(2)$
 $\downarrow$
 $= 4 * 3 * 2 * fact(1)$
 $\downarrow$
 $= 4 * 3 * 2 * 1$
 $\downarrow$
 $= 24$

Code (Java)

```
int fact(int n) {
    if (n == 0 || n == 1)
        return 1;
    return n * fact(n - 1);
}
```

Time Complexity : $O(n)$
 Space Complexity : $O(n)$

2. Fibonacci Series

Example / Dry Run

$n = 5$
 $fib(5) = fib(4) + fib(3)$
 $\swarrow \quad \searrow$
 $fib(4) \quad fib(3)$
 $\swarrow \searrow \quad \swarrow \searrow$
 $fib(3) \quad fib(2) \quad fib(2) \quad fib(1)$
 $\downarrow \downarrow \downarrow \downarrow$
 $0 \quad 1 \quad 2 \quad 3 \quad 5$

Code (Java)

```
int fib(int n) {
    if (n <= 1)
        return n;
    return fib(n - 1) +
           fib(n - 2);
}
```

Time Complexity : $O(2^n)$
 Space Complexity : $O(n)$

3. Sum of First N Natural Numbers

Example / Dry Run

$n = 4$
 $sum(4) = 4 + sum(3)$
 $\downarrow$
 $= 4 + 3 + sum(2)$
 $\downarrow$
 $= 4 + 3 + 2 + sum(1)$
 $\downarrow$
 $= 4 + 3 + 2 + 1$
 $\downarrow$
 $= 10$

Code (Java)

```
int sum(int n) {
    if (n == 0)
        return 0;
    return n + sum(n - 1);
}
```

Time Complexity : $O(n)$
 Space Complexity : $O(n)$

4. Print Numbers from 1 to N

Example / Dry Run

$n = 5$
 $print(1)$
 $\downarrow$
 $print(2)$
 $\downarrow$
 $print(3)$
 $\downarrow$
 $print(4)$
 $\downarrow$
 $print(5)$

Code (Java)

```
void print(int i, int n) {
    if (i > n) return;
    System.out.print(i + " ");
    print(i + 1, n);
}
```

Time Complexity : $O(n)$
 Space Complexity : $O(n)$

5. Print Numbers from N to 1

Example / Dry Run

$n = 5$
 $print(5)$
 $\downarrow$
 $print(4)$
 $\downarrow$
 $print(3)$
 $\downarrow$
 $print(2)$
 $\downarrow$
 $print(1)$

Code (Java)

```
void print(int n) {
    if (n < 1) return;
    System.out.print(n + " ");
    print(n - 1);
}
```

Time Complexity : $O(n)$
 Space Complexity : $O(n)$

6. Power (x^n)

Example / Dry Run

$x = 2, n = 4$
 $pow(2, 4) = 2 * pow(2, 3)$
 $\downarrow$
 $= 2 * 2 * pow(2, 2)$
 $\downarrow$
 $= 2 * 2 * 2 * pow(2, 1)$
 $\downarrow$
 $= 2 * 2 * 2 * 2$
 $\downarrow$
 $= 16$

Code (Java)

```
int pow(int x, int n) {
    if (n == 0)
        return 1;
    return x * pow(x, n - 1);
}
```

Time Complexity : $O(n)$
 Space Complexity : $O(n)$

3. Recursion on Arrays

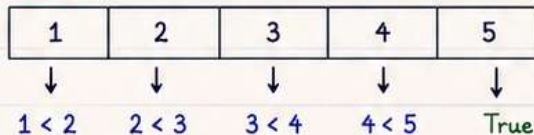
@AmanViews

These problems show how recursion can be applied on arrays to solve different types of questions.

1. Check if Array is Sorted

Example : arr = [1, 2, 3, 4, 5]

Dry Run :



Code (Java)

```
boolean isSorted(int[] arr, int i) {  
    if (i == arr.length - 1) return true;  
    if (arr[i] > arr[i + 1]) return false;  
    return isSorted(arr, i + 1);  
}
```

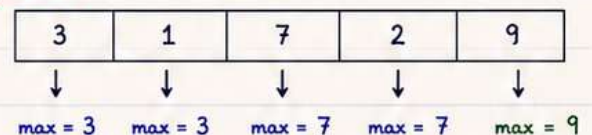
Time Complexity : $O(n)$

Space Complexity : $O(n)$

2. Find Maximum Element

Example : arr = [3, 1, 7, 2, 9]

Dry Run :



Code (Java)

```
int findMax(int[] arr, int i) {  
    if (i == arr.length - 1) return arr[i];  
    int maxFromRest = findMax(arr, i + 1);  
    return Math.max(arr[i], maxFromRest);  
}
```

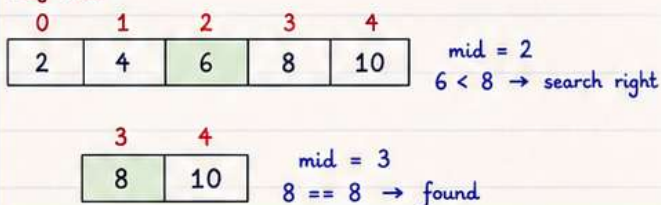
Time Complexity : $O(n)$

Space Complexity : $O(n)$

3. Binary Search (Recursive)

Example : arr = [2, 4, 6, 8, 10], target = 8

Dry Run :



Code (Java)

```
int binarySearch(int[] arr, int l, int r, int target) {  
    if (l > r) return -1;  
    int mid = l + (r - l) / 2;  
    if (arr[mid] == target) return mid;  
    if (arr[mid] > target) return  
        binarySearch(arr, l, mid - 1, target);  
    return binarySearch(arr, mid + 1, r, target);  
}
```

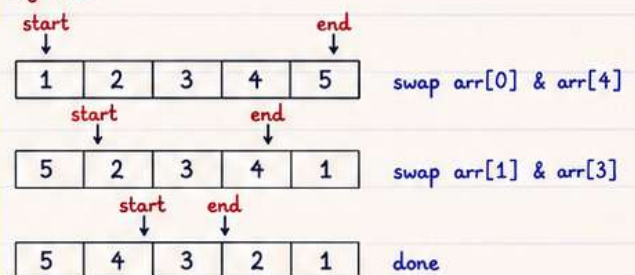
Time Complexity : $O(\log n)$

Space Complexity : $O(\log n)$

4. Reverse Array

Example : arr = [1, 2, 3, 4, 5]

Dry Run :



Code (Java)

```
void reverse(int[] arr, int start, int end) {  
    if (start >= end) return;  
    int temp = arr[start];  
    arr[start] = arr[end];  
    arr[end] = temp;  
    reverse(arr, start + 1, end - 1);  
}
```

Time Complexity : $O(n)$

Space Complexity : $O(n)$

4. Recursion on Strings

Recursion can be used on strings to solve problems like reversing, checking palindrome, counting characters, etc.

1. Reverse a String

Example : str = "hello"

Output : "olleh"

Dry Run :

```
reverse("hello")
= 'h' + reverse("ello")
= 'h' + 'e' + reverse("llo")
= 'h' + 'e' + 'l' + reverse("lo")
= 'h' + 'e' + 'l' + 'l' + reverse("o")
= 'h' + 'e' + 'l' + 'l' + 'o' + reverse("")
= 'h' + 'e' + 'l' + 'l' + 'o' + ''
= "olleh"
```

Code (Java)

```
String reverse (String s) {
    if (s.length() == 0)
        return "";
    return reverse(s.substring(1))
        + s.charAt(0);
}
```

Time Complexity : $O(n)$

Space Complexity : $O(n)$

(Recursion Stack)

2. Check Palindrome String

Example : str = "madam"

Output : true

Dry Run :

```
check("madam", 0, 4)
↓
m == m → true
↓
check("madam", 1, 3)
↓
a == a → true
↓
check("madam", 2, 2)
↓
i >= j → true
↓
return true
```

Code (Java)

```
boolean isPalindrome (String s,
                        int i, int j) {
    if (i >= j) return true;
    if (s.charAt(i) != s.charAt(j))
        return false;
    return isPalindrome(s, i + 1, j - 1);
}
```

Time Complexity : $O(n)$

Space Complexity : $O(n)$

(Recursion Stack)

3. Count Occurrences of a Character

Example : str = "programming", ch = 'r'

Output : 2

Dry Run :

```
count("programming", 'r', 0)
↓
'p' != 'r' → 0 + count(1)
↓
'r' == 'r' → 1 + count(2)
↓
'o' != 'r' → 0 + count(3)
↓
...
↓
count("", 'r', n) → 0
↓
Total = 2
```

Code (Java)

```
int countOcc (String s, char ch, int i) {
    if (i == s.length())
        return 0;
    int match = (s.charAt(i) == ch) ? 1 : 0;
    return match + countOcc(s, ch, i + 1);
}
```

Time Complexity : $O(n)$

Space Complexity : $O(n)$

(Recursion Stack)

4. Remove All Occurrences of a Character

Example : str = "abbaca", ch = 'a'

Output : "bbc"

Dry Run :

```
remove("abbaca", 'a', 0)
↓
'a' → skip, go to 1
↓
'b' → keep → "b" + ...
↓
'b' → keep → "bb" + ...
↓
'a' → skip
↓
'c' → keep → "bbc" + ...
↓
'a' → skip
↓
end → ""
↑
Result = "bbc"
```

Code (Java)

```
String removeChar (String s, char ch, int i) {
    if (i == s.length())
        return "";
    if (s.charAt(i) == ch)
        return removeChar(s, ch, i + 1);
    else
        return s.charAt(i) +
            removeChar(s, ch, i + 1);
}
```

Time Complexity : $O(n)$

Space Complexity : $O(n)$

(Recursion Stack)

5. Recursion on Combinations & Subsequences

These problems are very important in interviews.

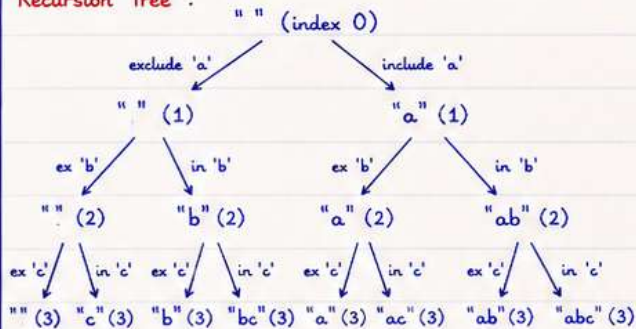
They follow the pattern of "Choose or Skip".

1. Generate All Subsequences

Example : str = "abc"

Output : ["", "a", "b", "c", "ab", "ac", "bc", "abc"]

Recursion Tree :



Code (Java)

```
void subseq(String s, int index, String curr) {
    if (index == s.length()) {
        System.out.println(curr);
        return;
    }
    // Exclude current character
    subseq(s, index + 1, curr);
    // Include current character
    subseq(s, index + 1, curr + s.charAt(index));
}
```

Time Complexity : $O(2^n)$

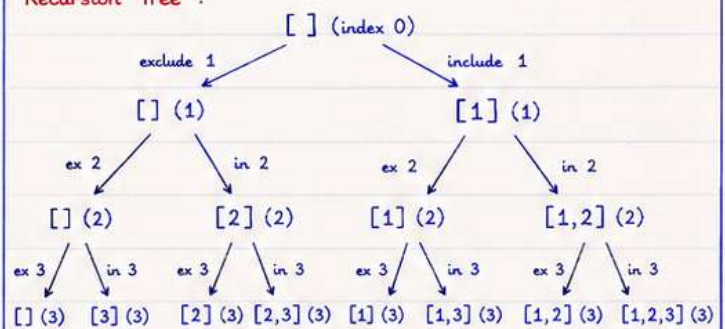
Space Complexity : $O(n)$
(Recursion Stack)

2. Generate All Subsets of a Set

Example : arr = [1, 2, 3]

Output : [], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]

Recursion Tree :



Code (Java)

```
void subsets(int[] arr, int index, List<Integer> curr) {
    if (index == arr.length) {
        System.out.println(curr);
        return;
    }
    // Exclude current element
    subsets(arr, index + 1, curr);
    // Include current element
    curr.add(arr[index]);
    subsets(arr, index + 1, curr);
    curr.remove(curr.size() - 1); // backtrack
}
```

Time Complexity : $O(2^n)$

Space Complexity : $O(n)$
(Recursion Stack)

3. Generate All String Combinations

Example : str = "abc"

Output : ["a", "b", "c", "ab", "ac", "bc", "abc"]

(Without empty string)

Code (Java)

```
void stringComb(String s, int index, String curr) {
    if (index == s.length()) {
        if (!curr.isEmpty()) System.out.println(curr);
        return;
    }
    // Exclude current character
    stringComb(s, index + 1, curr);
    // Include current character
    stringComb(s, index + 1, curr + s.charAt(index));
}
```

Time Complexity : $O(2^n)$

Space Complexity : $O(n)$
(Recursion Stack)

4. Phone Keypad Letter Combinations

Example : digits = "23"

Mapping : 2 → "abc", 3 → "def"

Output : ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"]

Code (Java)

```
String[] map = {"", "", "abc", "def", "ghi",
               "jkl", "mno", "pqrs", "tuv", "wxyz"},
void keypad(String digits, int index, String curr) {
    if (index == digits.length()) {
        System.out.println(curr);
        return;
    }
    int d = digits.charAt(index) - '0';
    String letters = map[d];
    for (int i = 0; i < letters.length(); i++) {
        keypad(digits, index + 1, curr + letters.charAt(i));
    }
}
```

Time Complexity : $O(3^n * 4^m)$

(n = digits with 3 letters,
m = digits with 4 letters)

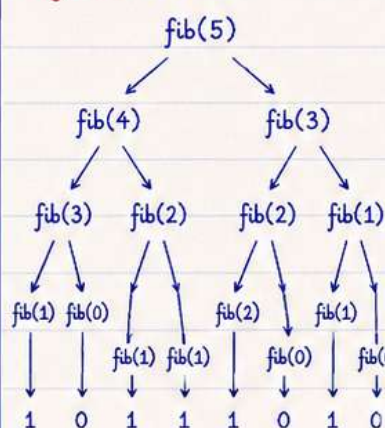
Space Complexity : $O(n)$
(Recursion Stack)

5. Nth Fibonacci Number

Find the Nth Fibonacci number using recursion.

Fibonacci Series : 0, 1, 1, 2, 3, 5, 8, 13, ...

Dry Run : n = 5



Code (Java)

```
int fib(int n) {
    if (n <= 1)
        return n;
    return fib(n-1) +
           fib(n-2);
}
```

Time Complexity : $O(2^n)$

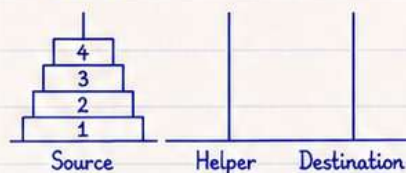
(Exponential)

Space Complexity : $O(n)$

(Recursion Stack)

6. Tower of Hanoi

Move N disks from source rod to destination rod using helper rod.



Steps for N = 3 :

1. Move disk 1 from Source to Destination
2. Move disk 2 from Source to Helper
3. Move disk 1 from Destination to Helper
4. Move disk 3 from Source to Destination
5. Move disk 1 from Helper to Source
6. Move disk 2 from Helper to Destination
7. Move disk 1 from Source to Destination

Code (Java)

```
void tower(int n, char s,
           char h, char d) {
    if (n == 1) {
        System.out.println(
            "Move disk " + n +
            " from " + s +
            " to " + d);
        return;
    }
    tower(n-1, s, d, h);
    tower(1, s, h, d);
    tower(n-1, h, s, d);
}
```

Time Complexity : $O(2^n)$

Space Complexity : $O(n)$

(Recursion Stack)

7. Check if String is a Palindrome (Recursive)

Example : str = "madam" Output : true

Dry Run :

```
check(0, 4)
m == m → check(1, 3)
a == a → check(2, 2)
d == d → check(3, 1)
3 >= 1 → true
```

Code (Java)

```
boolean isPalindrome(String s,
                     int i, int j) {
    if (i >= j) return true;
    if (s.charAt(i) != s.charAt(j))
        return false;
    return isPalindrome(s, i+1, j-1);
}
```

Time Complexity : $O(n)$

Space Complexity : $O(n)$

(Recursion Stack)

8. Sum of Digits of a Number

Example : n = 1234 Output : 10

$$1 + 2 + 3 + 4 = 10$$

Dry Run : n = 1234

```
sum(1234)
= 4 + sum(123)
= 4 + 3 + sum(12)
= 4 + 3 + 2 + sum(1)
= 4 + 3 + 2 + 1 + sum(0)
= 10
```

Code (Java)

```
int sumDigits(int n) {
    if (n == 0) return 0;
    return (n % 10) +
           sumDigits(n / 10);
}
```

Time Complexity : $O(d)$

(d = number of digits)

Space Complexity : $O(d)$

(Recursion Stack)

These problems build strong recursion skills and are commonly asked in technical interviews.

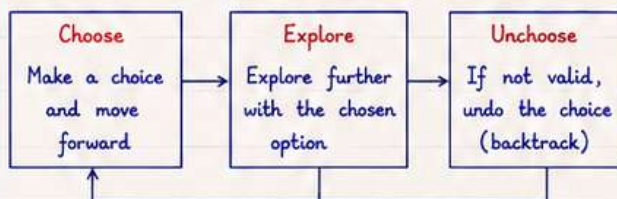
BACKTRACKING

Backtracking is an algorithmic technique for solving problems incrementally by trying out options and abandoning ("backtracking") those that fail to lead to a solution.

1. What is Backtracking?

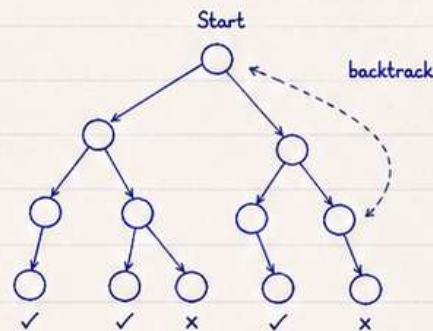
- It builds solution step by step.
- If a choice doesn't lead to a valid solution, we undo it and try the next choice.
- It uses recursion + undo (backtrack) to explore all possible options.

3. Backtracking Process (Choose → Explore → Unchoose)



4. State Space Tree

- All possible choices form a tree.
- Each node represents a state.
- Backtracking explores paths from root to leaf.
- If a path is invalid, we backtrack to previous node and try another path.



7. Famous Backtracking Problems

1. N-Queens Problem

Place N queens on an $N \times N$ chessboard such that no two queens attack each other.

Example ($N = 4$):

.	Q	.	.
.	.	.	Q
Q	.	.	.
.	.	Q	.

Idea:

- Place one queen in each row.
- Check column and both diagonals.
- If safe, place queen and move to next row.
- If not possible, backtrack.

Code (Pseudo):

```
solve(row):
    if row == N → return true
    for col in 0 to N-1:
        if isSafe(row, col):
            place queen
            if solve(row + 1): return true
            remove queen (backtrack)
    return false
```

2. Sudoku Solver

Fill a 9×9 grid so that each row, column and 3×3 subgrid contains numbers 1 to 9 exactly once.

Idea:

- Find an empty cell.
- Try numbers 1 to 9.
- If valid, place and move forward.
- If not, backtrack.

5	3	.	.	7	.	.	.
6	.	.	1	9	5	.	.
.	9	8	.	.	6	.	.
8	.	.	.	6	.	.	3
4	.	.	8	.	3	.	1
7	.	.	.	2	.	.	6
.	6	.	.	.	2	8	.
.	.	.	4	1	9	.	5
.	.	.	.	8	.	7	9

Code (Pseudo):

```
solve():
    find empty cell (r, c)
    if none → return true
    for num in 1 to 9:
        if isSafe(r, c, num):
            board[r][c] = num
            if solve(): return true
            board[r][c] = 0 // backtrack
    return false
```

3. Subsets (Power Set)

Given a set of distinct integers, return all possible subsets (power set).

Example: [1, 2, 3]

Output: [], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]

Idea:

- At each index, decide:
 - Pick the element
 - Not pick the element
- Explore all possibilities.

Code (Pseudo):

```
solve(index):
    if index == n:
        add current subset to result
        return
    pick nums[index]
    solve(index + 1)
    not pick nums[index]
    solve(index + 1)
```

4. Permutations

Given an array of distinct integers, return all possible permutations.

Example: [1, 2, 3]

Output: [1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]

Idea:

- Fix one element at a time.
- Swap with remaining elements.
- Recursively generate for the rest.
- Backtrack (swap back).

Code (Pseudo):

```
solve(index):
    if index == n:
        add current permutation to result
        return
    for i in index to n-1:
        swap(arr[index], arr[i])
        solve(index + 1)
        swap(arr[index], arr[i]) // backtrack
```

5. Combination Sum

Given candidates (distinct) and target, find all unique combinations where numbers sum to target. (Each number can be used unlimited times)

Example: candidates = [2, 3, 6, 7], target = 7

Output: [[2, 2, 3], [7]]

Idea:

- Choose a number.
- Reduce target by that number.
- Stay at same index (repetition allowed).
- Backtrack and try next number.

Code (Pseudo):

```
solve(index, target):
    if target == 0:
        add current combination to result
        return
    for i in index to n-1:
        if candidates[i] <= target:
            pick candidates[i]
            solve(i, target - candidates[i])
            backtrack
```

6. Word Search (Exist)

Given a 2D board of characters and a word, find if the word exists in the grid. (Path can be horizontal or vertical, no cell can be used more than once)

Idea:

- Start DFS from each cell.
- Match next character in 4 directions.
- Mark visited cells.
- Backtrack if path breaks.

Code (Pseudo):

```
exists(i, j, index):
    if index == len(word): return true
    if out of bounds or visited or board[i][j] != word[index]:
        return false
    mark visited
    search 4 directions
    unmark visited (backtrack)
    return false
```


8. Advanced Backtracking Problems

1. Graph Coloring Problem

Given an undirected graph and M colors, determine if it is possible to color all vertices such that no two adjacent vertices have the same color.

Example:

Vertices = 4, Edges = {(0,1), (0,2), (1,2), (1,3)}

M = 3 Output: true

Idea:

- Assign a color to one vertex at a time.
- Check if the color is safe for current vertex.
- If safe, proceed to next vertex.
- If not, try another color.

Code (Pseudo):

```
color[0...V-1] = 0
boolean graphColoring(int v) {
    if (v == V) return true;
    for (int c = 1; c <= M; c++) {
        if (isSafe(v, c)) {
            color[v] = c;
            if (graphColoring(v + 1)) return true;
            color[v] = 0; // backtrack
        }
    }
    return false;
}

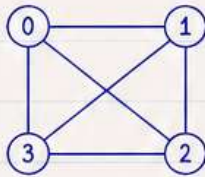
boolean isSafe(int v, int c) {
    for (each neighbor u of v)
        if (color[u] == c) return false;
    return true;
}
```

2. Hamiltonian Path / Cycle

Find a path in a graph that visits each vertex exactly once. If it ends at the starting vertex, it is a cycle.

Example:

Graph:



Output: One Hamiltonian Cycle → 0 1 2 3 0

Idea:

- Fix starting vertex.
- Try every unvisited vertex as next step.
- If all vertices are included, check if last is connected to first (for cycle).
- If not, backtrack.

Code (Pseudo):

```
path[0] = 0;
boolean hamCycle(int pos) {
    if (pos == V) {
        return isEdge(path[pos-1], path[0]);
    }
    for (int v = 1; v < V; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;
            if (hamCycle(pos + 1)) return true;
            path[pos] = -1; // backtrack
        }
    }
    return false;
}

boolean isSafe(int v, int pos) {
    if (!isAdjacent(path[pos-1], v)) return false;
    for (int i = 0; i < pos; i++)
        if (path[i] == v) return false;
    return true;
}
```

3. M-Coloring Problem

Similar to graph coloring, but optimized using constraint pruning (backtracking + forward checking).

Example:

Graph with V = 5, M = 3

Output: true / false

Idea:

- Assign colors one vertex at a time.
- Use forward checking to prune invalid colors early.
- If dead end, backtrack and try another color.

Code (Pseudo):

```
boolean mColoring(int v) {
    if (v == V) return true;
    for (int c = 1; c <= M; c++) {
        if (isSafe(v, c)) {
            color[v] = c;
            if (mColoring(v + 1)) return true;
            color[v] = 0; // backtrack
        }
    }
    return false;
}
```

4. Knight's Tour Problem

A knight on an N × N chessboard must visit every cell exactly once.

Example: N = 5

Output: One possible tour

1	12	17	4	9
16	5	2	11	18
7	22	13	8	3
6	15	24	19	10
21	20	23	14	25

Idea:

- Try all 8 possible knight moves.
- Move to next step if cell is safe.
- If all cells visited, solution found.
- Else backtrack.

Code (Pseudo):

```
board[r][c] = movei;
if (movei == N*N) return true;
for (each of 8 possible moves (x, y)) {
    if (isSafe(x, y)) {
        if (knightTour(x, y, movei + 1)) return true;
    }
}
board[r][c] = -1; // backtrack
return false;
}
```


COMMENT PDF

— TO GET —

PDF NOTES



Aman Views

Java • SQL • System Design • DSA
Backend Development • Web Development
QA Testing

Handwritten Notes



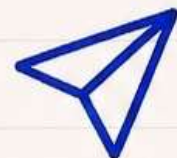
TO GET LINK OF ALL OUR NOTES,
CHECK THE LINK IN BIO.



SAVE



FOLLOW



SHARE